ChatGPT

Suitunes V1 — Versioned Feature Specification and Implementation Documentation

Specification version: Suitunes V1.0.0-dev-spec
Document status: Development single source of truth
Date: November 4, 2026
Primary implementation: C++ / JUCE
Primary test host: FL Studio
Initial plug-in targets: VST3 on Windows and macOS; Audio Unit on macOS
Product principle: Suitunes is a vocal pitch-correction plug-in built around two deliberately different workflows: **Auto Tune** for immediate real-time correction and **Fine Tune** for a friendlier, simplified offline pitch editor. It should not attempt to reproduce every capability of NewTone or Melodyne.

Executive summary

Suitunes V1 should be developed as two related pitch-correction systems sharing one musical model, preset/state system, and visual language. **Auto Tune** processes a monophonic vocal in real time, determines the singer's fundamental pitch, chooses an allowed target according to key/scale or incoming MIDI, smooths the requested correction, then applies a pitch shift while attempting to preserve vocal character. **Fine Tune** analyzes a recorded WAV, turns the vocal into visually understandable note regions and pitch curves, and lets the user automatically correct either a highlighted region or the entire vocal with a prominent **Auto Fine Tune** command. Users who want intervention can drag notes vertically, split a detected note, and undo their edits, but the interface is intentionally less precise and less tool-heavy than NewTone.

This division mirrors a well-established distinction in commercial pitch products. Image-Line describes Pitcher as a real-time pitch-correction/manipulation plug-in and NewTone as the related detailed editing tool. Pitcher also documents MIDI-controlled target pitches, correction-speed behavior, monophonic-source requirements, and formant preservation, while NewTone exposes note blocks, a detected continuous pitch line, note splitting, selections, and audio-to-MIDI-style score output. [1] Suitunes should adopt the *workflow lessons* from those tools without copying their UI.

The strongest first implementation candidate for real-time pitch shifting is **Signalsmith Stretch**. It is a C++ library under the permissive MIT license, exposes input/output latency explicitly, supports continuously changing pitch, and provides formant-related controls. Its current header identifies version 1.3.2, and the repository was active in October 2026. [2] **Rubber Band 4.0** is technically mature and has a dedicated `RubberBandLiveShifter`, but its own documentation states that live-shifter delay is typically **50 ms or more**, making it unsuitable as the first choice for a low-latency tracking mode; proprietary distribution also requires a commercial license unless Suitunes itself complies with the GPL. [3] **SoundTouch** is mature and can run in real time, but its time-stretch path documents latency of roughly **100 ms**, so it should be retained as a benchmark/fallback experiment rather than the default real-time vocal engine. [4]

For pitch detection, the first prototype should implement **YIN directly** and simultaneously benchmark **aubio YIN/YINFFT** as a reference. The original YIN paper describes a relatively simple F0 estimator designed for speech and musical signals that can be implemented efficiently and with low latency. [5] aubio conveniently provides `yin`, `yinfft`, `yinfast`, and several other estimators, but its code is GPLv3-or-later, so it should **not be embedded into a closed-source commercial Suitunes binary without an appropriate commercial licensing arrangement**. [6] The cleanest initial architecture is therefore **an in-house YIN implementation used in the shipping path**, with aubio serving as a development-time comparison oracle unless licensing is resolved.

The real-time goal needs to be treated as an engineering target, not a promise. Antares' current documentation provides useful competitive context: its low-latency modes are reported at approximately **0.77 ms to 2.3 ms at 48 kHz**, depending on mode, and the company recommends slower correction times for more natural vocal behavior. [7] Suitunes V1 should initially define a **recommended engineering target of ≤10 ms reported plug-in algorithmic latency**, with **≤15 ms as the provisional V1 ceiling**, while measuring complete monitored round-trip latency separately. This target was **not specified by the user** and must be considered provisional until prototype measurements show what is technically realistic.

At 48 kHz, 10 ms corresponds to **480 samples** and 15 ms to **720 samples**. At 44.1 kHz the same thresholds are approximately **441 and 662 samples**, and at 96 kHz they are **960 and 1,440 samples**. A host must be told the true algorithmic delay: JUCE exposes `AudioProcessor::setLatencySamples()`, VST3 requires processors to report group delay in samples, and FL Studio can automatically compensate correctly reported plug-in latency. [8]

The most important development rule is therefore: **prove the pitch engine before polishing the full product.** The first experiment should be a stripped-down JUCE VST3 using direct YIN + Signalsmith Stretch, instrumented for pitch error, correction latency, CPU use, and glitches. If that engine does not meet the provisional latency and quality targets, the architecture should permit swapping in another shifter without rewriting the musical logic or UI.

Product features and UI/UX specification

Product modes

Area	V1 requirement	Status
Auto Tune	Real-time monophonic vocal pitch correction	Required
Fine Tune	Offline/sample-based simplified pitch editor	Required
Key	Musical tonic shared by both modes	Required
Scale	Major, minor, chromatic, and user-enabled notes	Required
Speed	Controls how quickly correction approaches the target	Required

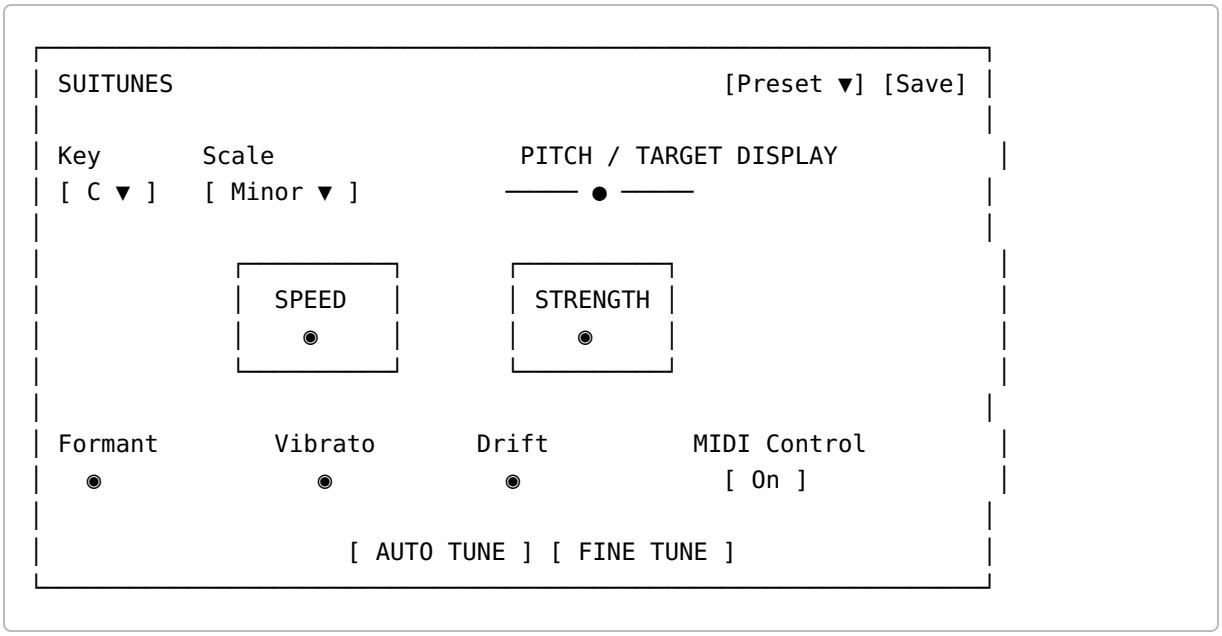
Area	V1 requirement	Status
Correction Strength	Controls how much correction is applied	Required
Formant control	Preserve/adjust apparent vocal character during shifting	Required; exact range **unspecified**
Vibrato handling	Preserve/reduce/enhance natural pitch movement	Required; exact controls **unspecified**
Pitch drift correction	Reduce slow sustained-note drift	Required
Pitch display	Show detected pitch and correction target	Required
MIDI target control	Incoming MIDI can define correction targets	Required
WAV import	Fine Tune accepts a vocal WAV	Required
WAV export	Fine Tune exports processed audio	Required
MIDI export	Fine Tune converts detected notes to MIDI	Required
MIDI import	Load a MIDI melody as correction guidance/reference	Required
Custom presets	Save/load Auto Tune settings	Required
`.sts` files	Portable Suitunes preset format	Required
Factory presets	None currently required	User preference: custom presets only
Polyphonic pitch correction	Chords/choirs/mixes	**Out of scope for V1**
Cloud processing	None required	**Out of scope unless later specified**

Pitcher itself explicitly expects a monophonic source, and Antares likewise advises using monophonic audio for reliable pitch detection, supporting Suitunes' V1 decision to make the correction engine voice/single-note focused rather than polyphonic. [9]

Auto Tune screen

The approved visual design is **simple**, not an engine simulation and not a dense precision editor. The base palette is **graphite gray and black with blue highlights**. The layout retains the approved dashboard-like organization, but **Speed** and **Correction Strength** must occupy the center of the interface and visually read as the two primary controls.

A conceptual layout is:



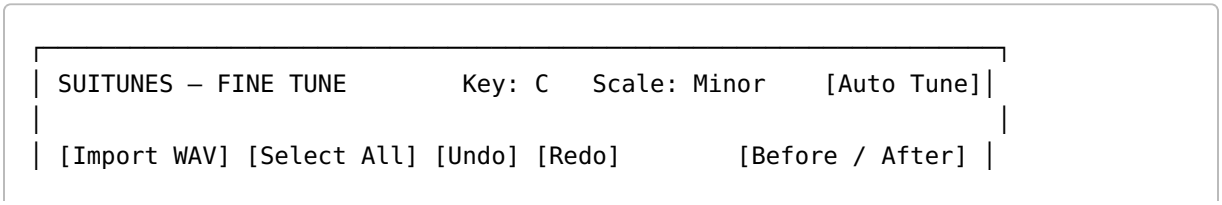
Speed is conceptually analogous to correction/retune speed in established tools: faster settings move the output toward the allowed pitch more aggressively, while slower settings preserve more of a singer's slides and legato behavior. Image-Line documents the same tradeoff for Pitcher, and Antares recommends slower retune values for natural correction and faster values for harder tuning. [10] The precise numerical range and knob mapping for Suitunes are **unspecified** and should not be frozen until listening tests.

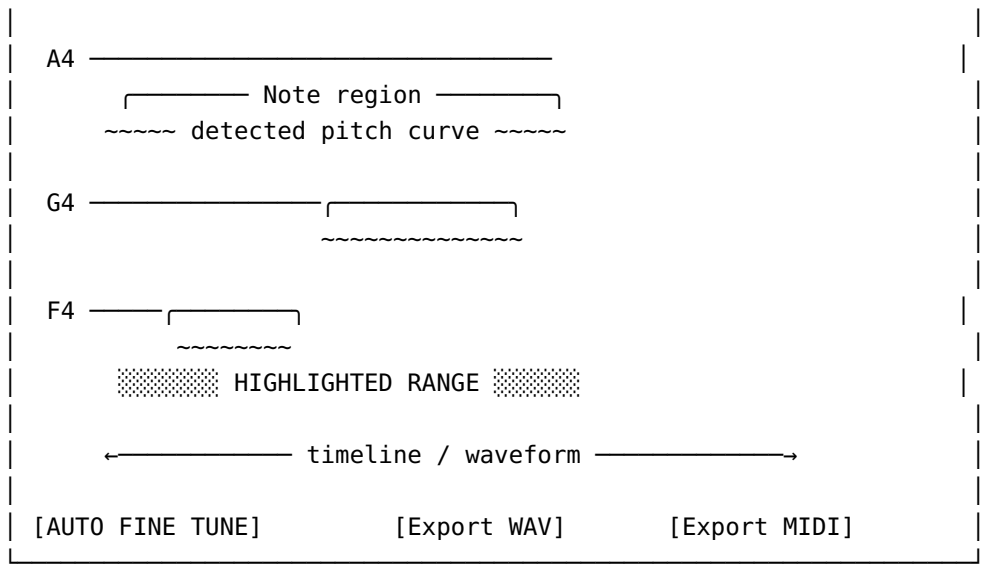
Correction Strength is a Suitunes-level control rather than a direct library parameter. It should interpolate between no correction and the full target trajectory generated by the tuning engine. Whether the UI exposes continuous 0–100%, named levels, or both is **unspecified**.

MIDI Control changes the target-note source. Incoming MIDI notes should override or constrain normal scale targeting according to a clearly defined mode. This is established behavior in Pitcher and Auto-Tune; Antares explicitly documents incoming MIDI as pitch targets, and Image-Line documents playing Pitcher's corrected target from a MIDI keyboard or piano roll. [11] **No-MIDI behavior must be explicit:** recommended V1 behavior is to bypass correction while MIDI Target mode is enabled and no target note exists, matching the current Antares behavior; this is a **recommended design choice, not a user-specified requirement**. [7]

Fine Tune screen

Fine Tune is a **simplified pitch editor**. It must communicate enough information for users to understand what was detected, but it should not resemble a full DAW piano roll or expose NewTone's many specialized controls.





The user can **click-drag over the timeline to create a range selection**. Only detected vocal material intersecting that selection is affected when **Auto Fine Tune** is pressed. **Select All** selects the entire imported vocal; Auto Fine Tune then processes the complete vocal. This region-based behavior is mandatory.

The **mouse wheel zooms horizontally around the pointer location**. Vertical scroll/zoom behavior is **unspecified**. Trackpad gestures, modifier keys, and minimum/maximum zoom levels are **unspecified** and should be assigned during implementation without changing the central behavior.

Each detected vocal note appears as a friendly block/region with a simplified pitch trace. NewTone demonstrates a useful representation consisting of an average note region plus a continuous detected pitch line, but Suitunes should visually simplify that concept rather than duplicate NewTone's piano-roll presentation. [12]

Manual operations are intentionally limited:

Interaction	V1 behavior
Drag note vertically	Change target semitone/pitch
Split	Divide one detected region at cursor position
Undo	Revert last edit/correction
Redo	Required because undo architecture should support it; UI position **unspecified**
Select range	Drag across timeline
Select All	Select complete imported vocal
Mouse wheel	Horizontal zoom centered near pointer

Interaction	V1 behavior
Auto Fine Tune	Correct selected material only
Before/After	Audition original versus processed
Delete note	**Unspecified**
Join notes	**Unspecified**
Timing drag	Previously discussed, but exact V1 behavior **unspecified**
Freehand pitch drawing	Not required
Detailed vibrato-envelope editor	Not required
Piano keyboard	Not required

NewTone provides far more manual controls—individual note snapping, formant edits, vibrato envelopes, transition editing, cuts, joins, time snapping, and pitch variation—which confirms that Suitunes can remain useful while intentionally omitting that complexity. [13]

Presets and `.sts`

Auto Tune must support user-created presets. `.sts` is the Suitunes portable preset extension.

Recommended V1 logical structure:

```
{
  "format": "SuitunesPreset",
  "formatVersion": 1,
  "productVersion": "1.0.0",
  "name": "My Vocal",
  "parameters": {
    "key": "C",
    "scale": "minor",
    "enabledNotes": [],
    "speed": 0.0,
    "correctionStrength": 0.0,
    "formant": 0.0,
    "vibrato": 0.0,
    "pitchDrift": 0.0,
    "midiControlMode": "off"
  }
}
```

The exact numeric ranges above are deliberately placeholders and therefore **unspecified**.
`formatVersion` is mandatory so a future Suitunes version can migrate older presets. Unknown fields

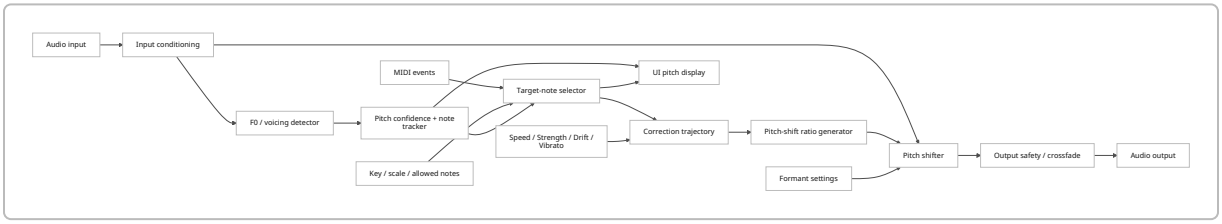
should be ignored when loading so V1 readers can tolerate forward-added metadata when safe. `.sts` should contain **settings only**, not the imported WAV, Fine Tune pitch cache, or edited audio.

Host project state is separate from `.sts`. JUCE's `AudioProcessorValueTreeState` is well suited to serializing plug-in parameters and state, but its state-copy and replace operations acquire locks and are explicitly documented as **not real-time safe**, so they must not occur inside `processBlock()`. [14]

Audio engine design and candidate evaluation

Real-time processing model

The intended Auto Tune signal path is:



The engine must separate **pitch estimation**, **musical target selection**, **correction trajectory generation**, and **audio pitch shifting**. This makes each component independently replaceable and testable.

A suggested internal representation is cents relative to A4 rather than directly manipulating Hz whenever correction curves are generated:

$$c = 1200 \log_2(f/440)$$

and a target-to-source shift ratio becomes:

$$r = 2^{\Delta c/1200}.$$

These equations are standard equal-tempered pitch relationships; implementation should use double precision for control calculations and float or double audio depending on the negotiated host path.

Pitch-detection candidates

Candidate	Latency characteristics	License	Maturity	Advantages	Drawbacks	V1 disposition
Direct YIN implementation	Window-dependent; original paper describes efficient, low-latency implementation	Our implementation	Established 2002 algorithm	Simple, interpretable; suited to speech/music; no external runtime dependency	Must implement confidence, octave-error mitigation and voicing logic ourselves	**Primary prototype**
aubio `yin`	Configurable buffer/hop; exact Suitunes latency must be measured	GPLv3+	Mature	Convenient baseline against known implementation	GPL incompatible with ordinary proprietary embedding without separate arrangement	Benchmark/ reference
aubio `yinfft`	Configurable; aubio default pitch method	GPLv3+	Mature	FFT-assisted implementation; straightforward A/B	Same licensing issue; spectral behavior may differ on weak fundamentals	Benchmark/ reference
aubio `yinfast`	Configurable	GPLv3+	Mature	Additional candidate with common API	Same licensing issue	Optional benchmark

aubio officially lists `yinfft`, `yin`, `yinfast`, `fcomb`, `mcomb`, `schmitt`, and `specacf`, with `yinfft` as its default pitch method. [15] The aubio project currently identifies itself as GPLv3-or-later. [16]

A detector should output at least:

```
struct PitchEstimate
{
    double frequencyHz;
    double midiFloat;
    float confidence;
    bool voiced;
```

```
    uint64_t analysisSample;
};
```

Required detector conditions: silence must not produce a valid target; unvoiced consonants should ordinarily pass without forced tonalization; octave jumps need hysteresis/continuity logic; and the detector must expose confidence so the target tracker can reject unreliable estimates. Exact confidence thresholds are **unspecified** and must be tuned empirically.

The original YIN algorithm uses modifications to autocorrelation-based fundamental-frequency estimation and reported materially lower error rates than competing methods in the paper's evaluation. [5] That historical result does not establish that YIN will outperform every modern detector on Suitunes vocals, so the project must benchmark rather than assume superiority.

Pitch-shifting candidates

Library	Relevant latency	License	Maturity/ current status	Pros	Cons	Recommendation
Signalsmith Stretch	Explicit `inputLatency()` and `outputLatency()`; exact configuration-dependent delay must be measured	MIT	Active; header currently identifies 1.3.2	C++11; permissive; dynamic pitch; formant support; clear latency API	Polyphonic/ spectral algorithm rather than vocal-specific PSOLA; may require tuning for monitoring latency	**First prototype**
Rubber Band 4.0 `LiveShifter`	Official docs: ~50 ms or more depending on configuration	GPL2+ or commercial	Mature; 4.0 adds live-specific API	Strong quality reputation; RT-safe processing path; formant-preserved modes	Latency too high for desired tracking feel; commercial license required for proprietary app	Fine Tune/offline experiment; not primary live path

Library	Relevant latency	License	Maturity/ current status	Pros	Cons	Recommendation
Rubber Band `Stretcher`	Configuration dependent; real-time and offline modes	GPL2+ or commercial	Mature	Excellent offline options; two-pass offline processing available	More integration complexity; same licensing issue	Candidate for offline Fine Tune render
SoundTouch	Time-stretch path around 100 ms according to official docs	LGPL 2.1 or commercial	Long-established; official changelog now lists 2.4.1	Very mature C++; portable; easy API	Latency unsuitable for premium vocal monitoring; WSOLA-like approach may not be ideal for dynamic vocal correction	Benchmark/ fallback only

Signalsmith documents separate input and output latency and explains how automation should be aligned with the internal processing time. It also supports formant compensation and recommends providing a rough fundamental-frequency estimate for its formant behavior—information Suitunes' detector can supply directly. [17]

Rubber Band 4.0's live shifter is explicitly designed as a simpler pitch-only API and is real-time safe in the sense of avoiding allocation, locking, and blocking in the normal processing path, but the official API documentation warns that its delay remains **50 ms or more**. [18] That essentially rules it out as the initial "feels like Pitcher" tracking engine even though it may be valuable for the offline Fine Tune renderer.

SoundTouch's own documentation places normal time-stretch processing near 100 ms latency and describes its core time-domain process as WSOLA-like. [19] That is substantially above the proposed Suitunes live target.

Correction trajectory

Raw detected F0 should **never be connected directly to the pitch-shifter ratio**. Doing so would follow detector jitter and create zippering or unstable tuning. The target controller should maintain:

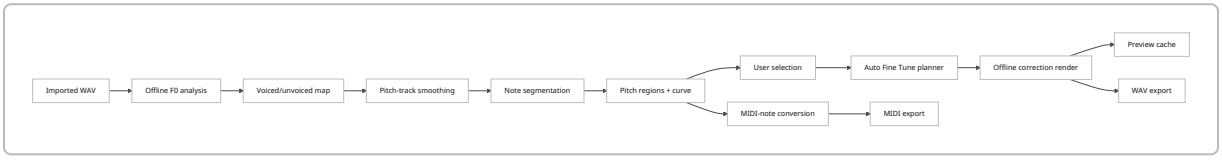
- voiced/unvoiced state;
- current stable source note;
- candidate destination note;

- hysteresis around note boundaries;
- target-note hold time;
- correction strength;
- retune/speed time;
- drift component;
- vibrato-residual component;
- MIDI target state.

Pitcher's documentation explicitly notes that fast correction sounds more artificial and slow correction sounds more natural with legato/glissando singing, and Antares similarly publishes retune-speed guidance for natural, expressive, and hard-tuned vocals. [10] Suitunes should therefore make Speed control the time constant or slew of the *target trajectory*, not the pitch detector itself.

Fine Tune analysis

Fine Tune can use a more computationally expensive multi-pass analyzer because it does not need to operate under live-monitoring latency. The analysis pipeline should:



Automatic segmentation should detect stable note regions, transitions, and likely separations rather than creating a new note for every short detector fluctuation. Exact segmentation thresholds, minimum note duration, breath detector, and transition rules are **unspecified** and must be tuned against vocal datasets.

DAW integration, MIDI, data formats, and plug-in state

Plug-in formats

JUCE can build VST3 audio/MIDI plug-ins and configure whether the plug-in receives and produces MIDI. [20] V1's required targets are:

Platform	Format	V1
Windows	VST3 64-bit	Required
macOS	VST3	Required
macOS	Audio Unit	Required
Windows	AAX	**Unspecified / not V1 requirement**
macOS	AAX	**Unspecified / not V1 requirement**
Linux	VST3	**Unspecified**
AUv3	**Unspecified**	

Apple documents Audio Units as the native plug-in architecture and supports Audio Unit processing and MIDI behavior on macOS. [21]

For VST3, MIDI note messages arrive through VST3 **event buses** rather than a legacy raw MIDI stream; Steinberg maps MIDI Note-On/Note-Off concepts to VST3 `NoteOnEvent` and `NoteOffEvent`. [22] JUCE's `MidiBuffer` abstraction should shield most of this format-specific handling from the DSP layer.

FL Studio MIDI routing

FL Studio's wrapper exposes plug-in MIDI input/output ports and permits matching port numbers between MIDI-generating channels and a plug-in. [23] Pitcher's own FL Studio documentation uses this exact model to route MIDI from a MIDI Out channel into Pitcher. [24] Therefore FL Studio acceptance testing must include:

1. Suitunes instantiated as a mixer effect.
2. MIDI input enabled in the VST3 build.
3. FL wrapper input port configured.
4. MIDI Out generator configured to the matching port.
5. Note-On/Note-Off verified sample-accurately enough for tuning.
6. Held-note and no-note behavior verified.
7. MIDI automation tested during playback and recording.

MIDI import and MIDI control

MIDI import should accept at minimum **Standard MIDI File Type 0 and Type 1**. The MIDI Association describes Standard MIDI Files as time-stamped event files and identifies Type 0 and Type 1 as the common file forms. [25]

When imported into Fine Tune, MIDI may be used as a **target melody**. The exact UI for mapping a MIDI track to the vocal is **unspecified**. Recommended V1 behavior is to read the first nonempty note track by default while providing a track picker if multiple note tracks exist.

Real-time MIDI Control should support:

- Note On;
- Note Off;
- overlapping held notes;
- changing targets while audio continues;
- incoming velocity available to the engine, though whether velocity changes tuning behavior is **unspecified**;
- MIDI pitch bend response **unspecified for V1**;
- MPE **not required**.

MIDI export

Fine Tune MIDI export should map each detected/edited vocal note to one MIDI note. Melodyne provides a useful established precedent: its exported MIDI preserves each audio note's **position, length, and pitch**,

derives MIDI velocity from amplitude, and omits breaths for vocal material. [26] Suitunes should adopt the same conceptual mapping.

Recommended V1 mapping:

Audio property	MIDI mapping
Edited target note	MIDI note number
Note start	MIDI Note-On time
Note end	MIDI Note-Off time
Relative note loudness	Velocity 1–127
Breath/unvoiced region	No note
Fine pitch curve/vibrato	Not exported in basic V1 SMF
Overlapping detected notes	Resolve to monophonic sequence
Tempo	Use host/import tempo if reliably known; otherwise **unspecified**
PPQ resolution	**Unspecified**

Because V1's source is explicitly monophonic, overlapping exported notes should normally be prevented. When segmentation produces overlap due to transition smoothing, the exporter should resolve it deterministically—recommended behavior is to end the earlier MIDI note at the later note's onset. This is **a proposed rule, not user-specified**.

Velocity should be computed from an amplitude statistic such as RMS or perceptual loudness over the note and normalized to MIDI's 1–127 velocity range. The exact curve is **unspecified**.

WAV export

Fine Tune must export the corrected region or full vocal as WAV. Required baseline:

- preserve source sample rate by default;
- preserve mono/stereo channel count where feasible;
- export at least 24-bit PCM and 32-bit float;
- ensure rendered length remains aligned with the original when no timing edits are made;
- include no unrequested normalization;
- handle head/tail latency from the selected shifter.

Signalsmith explicitly documents start/end pre-roll and flushing requirements, while Rubber Band's offline mode performs padding/delay compensation and supports exact start/duration processing. [27] The renderer must therefore compensate algorithmic head/tail padding before writing the WAV.

Dither behavior, metadata chunks, broadcast-WAV support, and sample-rate conversion are **unspecified**.

Host state versus `.sts`

Two separate persistence layers are mandatory:

Host state stores the entire state required for an FL Studio project to reopen identically. It should include Auto Tune parameters, mode selection, UI state where appropriate, and any Fine Tune editor metadata that can be safely embedded. How imported source audio is persisted inside a host project is **unspecified** and requires prototype testing because embedding long audio inside plug-in state may create excessively large DAW projects.

`.sts` **preset state** stores portable tuning settings only.

A minimal versioning contract is:

```
STS magic/signature: "SUITUNES_STS"
schema version: uint32 / logical integer
product version: semantic-version string
payload encoding: UTF-8 JSON or JUCE ValueTree binary/XML — final choice
unspecified
checksum: unspecified
optional future compression: unspecified
```

The preset loader must reject corrupt files gracefully and never execute data from presets.

JUCE/C++ architecture and real-time safety

A modular codebase is necessary so pitch-detector and shifter experiments can be swapped without destabilizing the plug-in.

```
Suitunes/
  Source/
    PluginProcessor.*
    PluginEditor.*

    dsp/
      IPitchDetector.h
      YinPitchDetector.*
      PitchTracker.*
      ScaleQuantizer.*
      CorrectionPlanner.*
      IPitchShifter.h
      SignalsmithShifter.*
      RubberBandShifter.*       // experimental
      SoundTouchShifter.*       // experimental
      FormantController.*
```

```
    AudioEngine.*

  finetune/
    OfflineAnalyzer.*
    NoteSegmenter.*
    FineTuneModel.*
    FineTuneRenderer.*
    SelectionModel.*
    UndoHistory.*

  midi/
    MidiTargetController.*
    MidiFileImporter.*
    MidiFileExporter.*

  state/
    ParameterLayout.*
    HostStateSerializer.*
    STSPresetSerializer.*
    STSMigrations.*

  ui/
    DashboardView.*
    FineTuneView.*
    PitchTimeline.*
    WaveformView.*
    SelectionOverlay.*
    Theme.*
```

Audio-thread rules

`processBlock()` is an audio-thread callback and JUCE explicitly warns against UI interaction from it. [28] The audio path must therefore:

- perform **no file I/O**;
- perform **no JSON/XML parsing**;
- avoid locks and blocking synchronization;
- avoid dynamic allocations in steady state;
- never call `AudioProcessorValueTreeState::copyState()` or `replaceState()` inside the callback because JUCE documents them as locking and not real-time safe; [14]
- preallocate all large buffers in `prepareToPlay()`;
- use lock-free or atomically readable control state between UI/control threads and DSP;
- post display information to the UI through a ring buffer or atomic snapshot.

JUCE notes that resizing `AudioBuffer` can cause allocation, so processing buffers should be prepared for the host's advertised maximum block size rather than expanded inside normal audio callbacks. [29]

Parameter smoothing

Any rapidly changing parameter that affects signal amplitude, pitch ratio, wet mix, or formant behavior must be smoothed. JUCE's `SmoothedValue` is explicitly intended to avoid glitches from abrupt value changes and supports linear and multiplicative smoothing. [30] JUCE's own plug-in tutorial likewise demonstrates gain ramping to prevent discontinuity clicks. [31]

Recommended rules:

- **pitch ratios:** multiplicative/log-domain smoothing;
- **wet/dry/gain:** linear smoothing;
- **Speed:** modifies the target trajectory rather than itself requiring sample-rate interpolation;
- **Correction Strength:** smooth changes over a short ramp;
- **key/scale changes:** crossfade or defer target reassignment to avoid instantaneous pitch discontinuities;
- **MIDI target switches:** allow fast but nonzero transition unless hard-tune mode intentionally demands immediate transitions.

Exact smoothing durations are **unspecified**.

Buffering

Support host block sizes at least 16, 32, 64, 128, 256, 512, 1,024, and variable block sequences. FL Studio's wrapper documentation confirms that hosts may use variable-size chunks and provides a compatibility option for plug-ins that improperly require fixed buffers. [23] Suitunes should **not depend on FL Studio's fixed-buffer workaround**.

Internal pitch-analysis hop sizes may differ from host blocks. A ring-buffer architecture should accumulate exactly the analysis window required by YIN and emit pitch estimates at a fixed hop independent of host callback boundaries.

Plug-in latency reporting

At prepare/configuration time:

```
const int algorithmicLatencySamples = engine.getLatencySamples();
setLatencySamples(algorithmicLatencySamples);
```

JUCE directs processors to call `setLatencySamples()` so the host knows the imposed delay. [32] VST3 defines this latency as processor group delay and requires hosts to be notified if it changes. [33]

Avoid changing the reported latency while transport is running unless absolutely necessary because the host may need to recompute compensation, potentially interrupting playback. [34] Prefer fixed latency per quality/mode setting.

Testing plan and feature acceptance criteria

Latency targets

These values are **recommended engineering targets; the user did not specify numeric latency thresholds**.

Sample rate	Preferred ≤10 ms	Provisional maximum ≤15 ms
44.1 kHz	441 samples	≈662 samples
48 kHz	480 samples	720 samples
88.2 kHz	882 samples	1,323 samples
96 kHz	960 samples	1,440 samples

These refer to **reported plug-in algorithmic latency only**. Interface input/output buffers, OS driver delay, converters, and DAW routing produce additional monitored round-trip latency and must be measured separately.

The competitive bar is substantially lower than 10 ms in some products: current Antares documentation advertises low-latency figures around 0.77–2.3 ms at 48 kHz depending on mode. [35] That should be treated as an aspirational external benchmark, not a guaranteed Suitunes V1 requirement.

FL Studio latency procedure

Use a project at 48 kHz and repeat at 44.1/96 kHz.

1. Route a sample-accurate impulse or sharp transient to a dry track and an identical Suitunes-processed track.
2. Disable correction or configure a unity/no-pitch-shift case while keeping the engine active.
3. Render both tracks.
4. Measure sample offset between input and processed waveform.
5. Compare measured offset to Suitunes' `getLatencySamples()`.
6. Verify FL Studio's wrapper displays the expected reported latency.
7. Enable Automatic Plugin Delay Compensation and verify the rendered processed track aligns with the dry reference.
8. Repeat at host buffer sizes 32/64/128/256/512/1024 where supported.
9. Test monitoring through the live microphone path separately; record loopback where possible to measure end-to-end latency.

FL Studio's wrapper exposes reported plug-in latency, and FL Studio's Automatic PDC uses detected plug-in latency to maintain mixer-track synchronization. [36]

Pitch-detection benchmark

Build a deterministic test corpus containing:

- clean male vocals;
- clean female vocals;
- low register;
- high register;
- breathy passages;
- strong vibrato;
- slides/glissando;
- consonant-heavy attacks;
- quiet note tails;
- noisy room recording;
- synthesized sine fundamentals for ground truth;
- harmonically rich synthetic voices with known F0.

For synthesized signals, measure cents error and octave-error rate exactly. For real vocals, compare Suitunes against manual annotation plus reference detectors.

Acceptance proposal:

Metric	Initial threshold
Stable synthetic tone median error	≤5 cents
Octave-error rate on clean synthetic sweep	<1%
False voiced detections on silence	0 sustained false notes
Real-vocal note identity	≥95% of annotated stable regions
Pitch display continuity	No unexplained octave jumps on accepted test corpus

These thresholds are **recommended and user-unspecified** and may be revised after the first corpus exists.

Real-time audio acceptance

Auto Tune is accepted only when all of the following are true:

- no crash or dropout in a 30-minute continuous playback/monitoring soak test;
- no heap allocation in the steady-state DSP path under profiler/instrumentation;
- no audible click when Speed or Strength moves continuously;
- key/scale change does not produce uncontrolled discontinuity;
- MIDI Note-On changes correction target predictably;
- MIDI Note-Off produces the documented no-target behavior;
- source silence does not turn into pitched output;
- reported latency matches measured latency within one processing block or a tighter documented margin;

- CPU remains comfortably below a single real-time core's deadline at normal buffer sizes;
- formant mode does not introduce catastrophic artifacts for ordinary ±3 semitone correction;
- bypass returns a level- and time-consistent signal after PDC.

CPU percentage thresholds are **unspecified** because the minimum supported processor has not been defined.

Fine Tune acceptance

Fine Tune is accepted when:

- WAV drag/import successfully creates a pitch analysis;
- note regions and pitch trace correspond audibly to the vocal;
- drag-selection highlights a clearly visible interval;
- mouse wheel zoom retains the pointer neighborhood;
- Auto Fine Tune modifies **only selected material**;
- Select All followed by Auto Fine Tune processes the full vocal;
- vertically dragging a note changes its correction target;
- Split creates two independently editable note regions without changing unedited audio;
- Undo returns the exact previous model state;
- repeated undo/redo does not progressively alter the waveform;
- Before/After audition switches reliably;
- export produces a time-aligned WAV with no unexplained head/tail offset;
- exported MIDI has the expected note pitch, onset, duration, and relative velocity.

MIDI acceptance

MIDI control is accepted when the same MIDI melody correctly controls Suitunes in FL Studio at minimum. Cross-host acceptance should subsequently include at least one additional Windows VST3 host and two macOS hosts covering VST3 and AU. Exact host names beyond FL Studio are **unspecified**.

MIDI export should be validated by re-importing the generated `.mid` into FL Studio and comparing note starts, ends, pitches, and velocities to the internal Fine Tune note model. Standard MIDI Files are explicitly intended for exchanging time-stamped MIDI data between programs. [37]

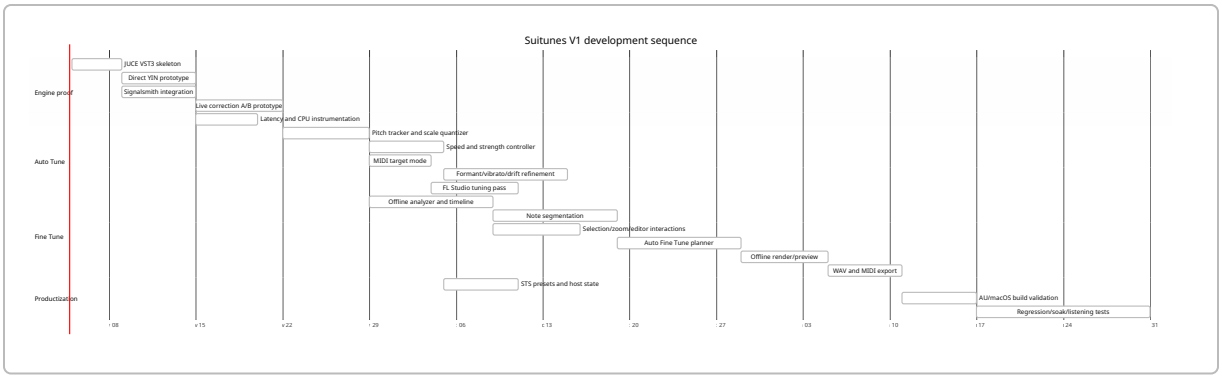
Preset acceptance

`.sts` is accepted when:

- save → modify plug-in → load produces an exact parameter round trip;
- presets survive application restart;
- unknown optional fields do not crash the loader;
- invalid JSON/data results in a clear load failure, not undefined state;
- schema version 1 is written on every V1 save;
- a migration-test fixture exists before schema version 2 is ever shipped;
- `.sts` files do not embed imported audio;
- DAW project save/reopen works even when no `.sts` has been explicitly saved.

Prioritized implementation roadmap, experiments, assets, and source appendix

Development roadmap



The calendar above is **planning scaffolding, not a delivery promise**. Relative task order matters more than the displayed dates.

Prioritized checklist

Priority	Task	Effort	Risk	Exit condition
P0	JUCE VST3 audio pass-through	Low	Low	Stable in FL Studio
P0	Implement direct YIN	Medium	Medium	Known-tone tests pass
P0	Integrate Signalsmith Stretch	Medium	Medium	Variable pitch ratio works without crash
P0	Measure shifter latency at all sample rates	Low	Low	Repeatable table exists
P0	Combine YIN + target selection + shifter	High	High	Live corrected vocal audible
P0	FL Studio monitoring test	Medium	High	No critical glitch/dropout
P1	Speed trajectory	Medium	Medium	Natural↔hard behavior audible
P1	Correction Strength	Low	Low	0→full interpolation works
P1	Key/scale/custom notes	Medium	Low	Deterministic target choices
P1	MIDI target mode	Medium	Medium	FL MIDI Out controls tuning
P1	Formant-preserved mode	High	High	Listening threshold met
P1	`.sts` presets	Medium	Low	Versioned round-trip works

Priority	Task	Effort	Risk	Exit condition
P2	Fine Tune WAV importer	Medium	Low	File loads and plays
P2	Offline pitch analysis	Medium	Medium	Stable timeline generated
P2	Note segmentation	High	High	Vocal notes are useful/editable
P2	Region selection + Select All	Medium	Low	Selection model tested
P2	Mouse-wheel zoom	Low	Low	Cursor-centered zoom works
P2	Auto Fine Tune selected region	High	High	Outside audio unchanged
P2	Manual drag/split/undo	Medium	Medium	Model edits reversible
P2	Before/After preview	Medium	Low	Immediate A/B works
P2	WAV export	Medium	Medium	Sample-aligned render
P2	MIDI export	Medium	Medium	Reimport matches model
P2	MIDI import target melody	Medium	Medium	Imported notes displayed/ usable
P3	Rubber Band offline comparison	Medium	Medium	Blind listening results documented
P3	SoundTouch benchmark	Low	Low	Latency/quality table documented
P3	Cross-DAW Windows validation	Medium	Medium	Additional host passes
P3	macOS VST3/AU validation	Medium	Medium	Both formats pass
P3	UI polish/accessibility/scaling	Medium	Low	Production UI accepted

Required next experiments

Experiment A — Signalsmith latency matrix. Configure 44.1, 48, and 96 kHz; test likely block sizes; call its latency-reporting APIs; verify with impulse measurements; then compare reported versus measured values. Signalsmith explicitly exposes separate input/output latency. [17]

Experiment B — YIN window/hop sweep. Test several F0 windows and hop sizes against the vocal corpus. Plot cents error, octave errors, CPU, and estimate age. The crucial tradeoff is low-frequency reliability versus reaction delay; the final values are **unspecified until measured**.

Experiment C — aubio cross-check. Run the same WAVs offline through aubio YIN and YINFFT and compare error patterns against Suitunes' direct YIN. aubio supports both algorithms through the same analysis interface. [15] Do not make aubio a shipping dependency unless licensing is explicitly resolved. [16]

Experiment D — Rubber Band live reality check. Measure `RubberBandLiveShifter::getStartDelay()` and confirm the documented ~50 ms+ delay under

vocal-oriented settings. [38] If confirmed, exclude it from Auto Tune tracking mode but retain it for Fine Tune blind listening.

Experiment E — Fine Tune shifter blind test. Take the same ten short vocal phrases, process identical predetermined correction curves with Signalsmith, Rubber Band offline, and any internal alternate, loudness-match results, and blind-rank naturalness, consonant integrity, vibrato, transient smearing, and formant quality.

Experiment F — FL Studio MIDI timing. Send quantized MIDI notes from FL Studio's MIDI Out into Suitunes while running a sustained vocal/test oscillator and log received note sample positions. FL Studio and Pitcher officially use matched MIDI ports for this style of control. [39]

Experiment G — end-to-end tracking test. Sing the same phrase through Suitunes and Pitcher at equal audio-device settings, record both outputs, and compare perceived monitoring delay, pitch-transition speed, artifacts, and CPU. This is the test that determines whether Suitunes actually *feels* competitive; source documentation alone cannot prove it.

Required UI assets

The asset package for V1 should contain:

Asset	Specification
Suitunes logo master	SVG preferred; transparent background; vector paths
Raster logo	PNG at 1×, 2×, 4×; transparent
App/plugin mark	Square simplified emblem, exact design **unspecified**
Undo/redo icons	Vector, monochrome/tintable
Split icon	Vector, simple
Select All icon	Optional; text button acceptable
Import icon	Vector
WAV export icon	Vector
MIDI export/import icon	Vector
MIDI control indicator	Blue active-state treatment
Pitch cursor	Vector/geometry
Note blocks	Drawn procedurally; no bitmap required
Pitch trace	Drawn procedurally
Selection overlay	Semi-transparent blue; exact opacity **unspecified**
Waveform	Drawn procedurally from cached peaks

The visual system is **graphite/black with blue highlights**. Orange is no longer the approved highlight. Fine Tune should share Suitunes typography and controls but remain visually lighter and friendlier than a full piano-roll editor.

Source appendix — prioritized primary references

The URLs below are intentionally included because this document is intended to serve as implementation documentation.

Priority A — direct implementation references

1. Signalsmith Stretch repository and documentation
   `https://github.com/Signalsmith-Audio/signalsmith-stretch`
   `https://signalsmith-audio.com/code/stretch/`
   Primary source for API, latency behavior, MIT license, formant facilities, and integration. [17]

2. YIN paper
   `https://doi.org/10.1121/1.1458024`
   Foundational reference for the proposed direct F0 detector. [40]

3. JUCE `AudioProcessor` documentation
   `https://docs.juce.com/master/classjuce_1_1AudioProcessor.html`
   Critical for `processBlock`, processor latency, state, and host integration. [41]

4. JUCE `AudioProcessorValueTreeState` documentation
   `https://docs.juce.com/master/classjuce_1_1AudioProcessorValueTreeState.html`
   Critical for host state/preset architecture and thread-safety constraints. [14]

5. Steinberg VST3 MIDI/event documentation
   `https://steinbergmedia.github.io/vst3_dev_portal/pages/Technical%2BDocumentation/About%2BMIDI/Index.html`
   Defines VST3 event-bus semantics for note events. [22]

6. Steinberg VST3 processor latency documentation
   `https://steinbergmedia.github.io/vst3_doc/vstinterfaces/classSteinberg_1_1Vst_1_1IAudioProcessor.html`
   Required for correct latency reporting. [34]

Priority B — alternative DSP implementations

1. Rubber Band documentation
   `https://www.breakfastquay.com/rubberband/`
   `https://www.breakfastquay.com/rubberband/integration.html`
   `https://www.breakfastquay.com/rubberband/code-doc/classRubberBand_1_1RubberBandLiveShifter.html`
   Compare live/offline rendering, latency, real-time safety, and licensing. [42]

2. SoundTouch documentation
   `https://soundtouch.surina.net/`
   `https://www.surina.net/soundtouch/README.html`
   Compare WSOLA-style processing, measured latency, and LGPL/commercial licensing. [43]

3. aubio analysis documentation and repository
   `https://aubio.org/manual/latest/py_analysis.html`
   `https://github.com/aubio/aubio`
   Reference implementation for YIN/YINFFT comparisons; shipping use requires license review. [6]

Priority C — product behavior and workflow references

1. FL Studio Pitcher manual
   `https://www.image-line.com/fl-studio-learning/fl-studio-online-manual/html/plugins/Pitcher.htm`
   Best primary reference for the target FL Studio real-time workflow, MIDI routing concepts, formant behavior, and correction speed. [24]

2. FL Studio NewTone manual
   `https://www.image-line.com/fl-studio-learning/fl-studio-online-manual/html/plugins/Newtone.htm`
   Reference for note/pitch visualization, selection, splitting, manual pitch edits, and score generation—**behavioral inspiration only, not UI copying**. [44]

3. FL Studio Plugin Wrapper documentation
   `https://www.image-line.com/fl-studio-learning/fl-studio-online-manual/html/plugins/wrapper.htm`
   Required for FL Studio MIDI, plug-in latency display, variable buffers, and wrapper testing. [23]

4. FL Studio PDC documentation
   `https://www.image-line.com/fl-studio-learning/fl-studio-online-manual/html/mixer_trackprops.htm`
   Required for latency-compensation verification. [45]

5. Antares MIDI-control documentation
   `https://help.antarestech.com/hc/en-us/articles/41109184245908-How-to-configure-MIDI-control-of-Auto-Tune-Pro-or-Auto-Tune-Artist-in-Logic-Pro`
   Reference for MIDI-driven correction behavior. [46]

6. Antares current tuning/latency guidance
   `https://help.antarestech.com/hc/en-us/articles/42858099043092-AutoTune-Best-Practices`
   Competitive reference for monitoring latency and correction-speed behavior. [35]

7. Celemony Melodyne Audio-to-MIDI documentation
   `https://helpcenter.celemony.com/M5/doc/melodyneAssistant5/en/`

`M5tour_ExportMIDI_standalone?env=standAlone`
Reference for note-position, duration, pitch, breath exclusion, and amplitude-to-velocity mapping. [26]

8. MIDI Association Standard MIDI Files reference
`https://midi.org/standard-midi-files`
Canonical basis for Suitunes `.mid` import/export. [37]

This V1 specification deliberately treats **Signalsmith + direct YIN as the first implementation to prove**, not as an irrevocable final engine. The acceptance gate is empirical: it must survive FL Studio testing, correctly report latency, respond naturally enough to real singing, preserve vocal character, and remain stable under real-time constraints. If it fails those tests, the `IPitchDetector` / `IPitchShifter` architecture allows the DSP implementation to change without changing the approved Suitunes feature set or UI contract.

---

[1] [9] [10] [11] [24] Pitcher
https://ww2.image-line.com/fl-studio-learning/fl-studio-online-manual/html/plugins/Pitcher.htm?utm_source=chatgpt.com

[2] [17] [27] GitHub - Signalsmith-Audio/signalsmith-stretch: C++ polyphonic pitch/time library (GitHub mirror) · GitHub
https://github.com/Signalsmith-Audio/signalsmith-stretch?utm_source=chatgpt.com

[3] Breakfast Quay
https://www.breakfastquay.com/news.html?utm_source=chatgpt.com

[4] [43] SoundTouch Sound Processing Library
https://soundtouch.surina.net/?utm_source=chatgpt.com

[5] YIN, a fundamental frequency estimator for speech and musica)
https://iro.umontreal.ca/~pift6080/H09/documents/papers/yin_pitch_tracker.pdf?utm_source=chatgpt.com

[6] [15] Analysis — aubio 0.4.9 documentation
https://aubio.org/manual/latest/py_analysis.html?utm_source=chatgpt.com

[7] AutoTune Advanced FAQ – AutoTune Support Center
https://help.antarestech.com/hc/en-us/articles/53453475818004-AutoTune-Advanced-FAQ?utm_source=chatgpt.com

[8] [32] [41] JUCE: juce::AudioProcessor Class Reference
https://docs.juce.com/master/classjuce_1_1AudioProcessor.html?utm_source=chatgpt.com

[12] NewTone
https://cluster.image-line.com/fl-studio-learning/fl-studio-online-manual/html/plugins/Newtone.htm?utm_source=chatgpt.com

[13] [44] NewTone
https://www.image-line.com/fl-studio-learning/fl-studio-online-manual/html/plugins/Newtone.htm?utm_source=chatgpt.com

[14] JUCE: juce::AudioProcessorValueTreeState Class Reference
https://docs.juce.com/master/classjuce_1_1AudioProcessorValueTreeState.html?utm_source=chatgpt.com

[16] aubio/README.md at master · aubio/aubio · GitHub
https://github.com/aubio/aubio/blob/master/README.md?plain=1&utm_source=chatgpt.com

18 38 Rubber Band Library: RubberBand::RubberBandLiveShifter Class Reference
https://breakfastquay.com/rubberband/code-doc/classRubberBand_1_1RubberBandLiveShifter.html?utm_source=chatgpt.com

19 SoundTouch library README
https://surina.net/soundtouch/README.html?utm_source=chatgpt.com

20 Create a basic Audio/MIDI plugin, Part 1 - JUCE
https://juce.com/tutorials/tutorial_create_projucer_basic_plugin/?utm_source=chatgpt.com

21 Incorporating Audio Effects and Instruments | Apple Developer Documentation
https://developer.apple.com/documentation/audiotoolbox/incorporating-audio-effects-and-instruments?utm_source=chatgpt.com

22 About MIDI in VST 3 - VST 3 Developer Portal
https://steinbergmedia.github.io/vst3_dev_portal/pages/Technical%2BDocumentation/About%2BMIDI/Index.html?utm_source=chatgpt.com

23 36 39 Plugin Wrapper
https://www.image-line.com/fl-studio-learning/fl-studio-online-manual/html/plugins/wrapper.htm?utm_source=chatgpt.com

25 37 Standard MIDI Files – MIDI.org
https://midi.org/standard-midi-files?utm_source=chatgpt.com

26 Audio to MIDI
https://helpcenter.celemony.com/M5/doc/melodyneAssistant5/en/M5tour_ExportMIDI_standalone?env=standAlone&utm_source=chatgpt.com

28 JUCE: juce::AudioProcessorGraph Class Reference
https://docs.juce.com/master/classjuce_1_1AudioProcessorGraph.html?utm_source=chatgpt.com

29 JUCE: juce::AudioBuffer< Type > Class Template Reference
https://docs.juce.com/develop/classjuce_1_1AudioBuffer.html?utm_source=chatgpt.com

30 JUCE: juce::SmoothedValue< FloatType, SmoothingType > Class Template Reference
https://docs.juce.com/master/classjuce_1_1SmoothedValue.html?utm_source=chatgpt.com

31 Adding plug-in parameters - JUCE
https://juce.com/tutorials/tutorial_audio_parameter/?utm_source=chatgpt.com

33 34 VST 3 Interfaces: IAudioProcessor Class Reference
https://steinbergmedia.github.io/vst3_doc/vstinterfaces/classSteinberg_1_1Vst_1_1IAudioProcessor.html?utm_source=chatgpt.com

35 AutoTune Best Practices – AutoTune Support Center
https://help.antarestech.com/hc/en-us/articles/42858099043092-AutoTune-Best-Practices?utm_source=chatgpt.com

40 YIN, a fundamental frequency estimator for speech and music - PubMed
https://pubmed.ncbi.nlm.nih.gov/12002874/?utm_source=chatgpt.com

42 Rubber Band Audio Time Stretcher Library
https://www.breakfastquay.com/rubberband/?utm_source=chatgpt.com

45 Mixer Track Effects
https://www.image-line.com/fl-studio-learning/fl-studio-online-manual/html/mixer_trackprops.htm?utm_source=chatgpt.com

46 How to configure MIDI control of Auto-Tune Pro (or Auto-Tune Artist) in Logic Pro – AutoTune Support Center
https://help.antarestech.com/hc/en-us/articles/41109184245908-How-to-configure-MIDI-control-of-Auto-Tune-Pro-or-Auto-Tune-Artist-in-Logic-Pro?utm_source=chatgpt.com